

Manasight+ Deep Dive Guide

What This Document Is

This guide is the "I vibecoded this and now I want to actually understand it" version of the project documentation.

It is written for a beginner programmer. I am going to use plain English first, then connect that explanation back to the actual files in the repo.

This guide does **not** explain every single Python function definition line by line. There are too many for one useful document. Instead, it explains:

- 1 each major function of the **project**,
- 2 which files own that function,
- 3 how data moves through the system,
- 4 which parts are the source of truth,
- 5 which parts are bridge code,
- 6 which parts are probably not aligned with your goals yet.

The One-Sentence Summary

Manasight+ watches MTG Arena's `Player . log`, turns raw log text into structured events, builds match and game facts from those events, saves local artifacts, and optionally sends analysis-ready rows to Google Sheets.

The Five Most Important Ideas

If you only remember five things, remember these:

- a fast live parser - a slower analytics/reporting sidecar

- 1 `Player . log` is the only raw source of truth.
- 2 The parser and app state layer decide what the log means.
- 3 Google Sheets is a transport and display layer, not the truth layer.
- 4 The project has two personalities:
- 5 Your biggest long-term complexity is not "parsing text." It is "keeping truth ownership clear across parser -> artifacts -> webhook -> workbook."

The Project Goals, In Plain English

Right now the project is trying to do several jobs at once:

- 1 Track matches and games accurately.
- 2 Understand cards being played in real MTGA gameplay.
- 3 Build per-card analytics like win rates.
- 4 Feed a Google Sheet for reporting.
- 5 Give you local operator/debugging tools.

Those goals are all reasonable, but they do not all want the same architecture.

Examples:

- live gameplay tracking wants low latency and low overhead
- Sheets reporting wants stable rows and low noise
- debugging wants lots of detail
- card analytics wants consistent identifiers and historical artifacts

That is why the project sometimes feels "busy." It is serving several different goals at once.

The Layered Architecture

The cleanest way to understand Manasight+ is as six layers:

1. MTGA raw log source
2. parser and state interpretation
3. local artifacts and transport
4. workbook landing sheets
5. helper/support formulas
6. dashboard/reporting tabs

Here is what each layer means.

1. MTGA raw log source

This is just Arena's `Player.log`.

The project does not control this file. Arena writes it. Manasight+ only reads it.

Key files:

- `src/manasight_parser_py/log/tailer.py`
- `src/manasight_parser_py/log/entry.py`
- `src/manasight_parser_py/stream.py`

2. parser and state interpretation

This layer turns raw log chunks into meaning.

It decides things like:

- "this chunk is a GameState"
- "this means a match started"

- "this was a mulligan keep"
- "this game belonged to match X"
- "this card moved from hand to stack"

Key files:

- `src/manasight_parser_py/router.py`
- `src/manasight_parser_py/events.py`
- `src/manasight_parser_py/parsers/`
- `src/manasight_parser_py/app/extractors.py`
- `src/manasight_parser_py/app/state.py`
- `src/manasight_parser_py/app/models.py`
- `src/manasight_parser_py/app/gameplay_actions.py`

3. local artifacts and transport

This layer writes things to disk and optionally posts them elsewhere.

Examples:

- daily JSONL match logs
- active deck files
- action logs
- card performance files
- webhook posts to Google Sheets

Key files:

- `src/manasight_parser_py/app/outputs.py`
- `src/manasight_parser_py/app/runtime_surfaces.py`
- `src/manasight_parser_py/app/analytics_sidecar.py`
- `src/manasight_parser_py/app/sheet_exports.py`
- `src/manasight_parser_py/app/diagnostics.py`

4. workbook landing sheets

These are the first tabs that receive posted rows.

Main ones:

- Match Log
- Game Log

Support/optional ones:

- Deck Snapshot

- Card Performance
- some legacy bridge/debug tabs

The workbook receiver is:

- `tools/google_apps_script/Code.gs`

5. helper/support formulas

These tabs help the workbook classify, normalize, or look up things.

Examples:

- Helper Table
- Tier Source Data

These should support the workbook, not redefine parser truth.

6. dashboard/reporting tabs

These are the user-facing summaries and comparisons.

Examples:

- Dashboard
- Experiments

These are display layers.

The Actual Runtime Flow

When the parser is running during a match, the system roughly does this:

```

Player.log changes
-> file tailer reads new bytes
-> line buffer groups lines into entries
-> router asks parser modules to recognize entries
-> parser returns typed event objects
-> state layer updates match/game truth
-> gameplay action layer extracts card actions
-> local JSONL log is written
-> analytics sidecar updates local artifacts
-> optional webhook rows are sent to Google Sheets

```

That flow is mostly owned by:

- `src/manasight_parser_py/app/runner.py`

`runner.py` is the real traffic cop of the project.

The Most Important Files To Learn First

If you want the shortest path to real understanding, start with these files in this order:

```
1  main.py
2  src/manasight_parser_py/app/runner.py
3  src/manasight_parser_py/app/state.py
4  src/manasight_parser_py/app/models.py
5  src/manasight_parser_py/app/extractors.py
6  src/manasight_parser_py/app/transforms.py
7  src/manasight_parser_py/app/gameplay_actions.py
8  src/manasight_parser_py/app/runtime_surfaces.py
9  src/manasight_parser_py/app/outputs.py
10 tools/google_apps_script/Code.gs
```

If you understand those ten, you understand the project far better than "I run a script and hope."

What Each Major Area Does

Entry Points and Operator Tools

`main.py`

Purpose:

- the simplest human-friendly way to start the parser

What it really does:

- makes sure `src/` is importable
- loads the package entry point
- starts the async runtime

Why it matters:

- this is the "front door" to the parser

`live_print_filtered_v11_match_summary.py`

Purpose:

- compatibility wrapper for older launch habits

Why it exists:

- it preserves an older entry point name

`tools/auto_launcher/manasight_launcher_auto.py`

Purpose:

- operator UI
- start/stop parser
- show health
- open local artifacts
- drive hand-confirmation tools

Important truth:

- this file is **not** the parser
- it is a local control panel for the parser

Current risk:

- the launcher can feel like "the app"
- but it is really a UI shell around the parser process
- if the launcher UI hangs, the parser may still be running correctly

That distinction matters a lot.

Raw Log Ingestion

`src/manasight_parser_py/log/tailer.py`

Purpose:

- read new bytes from `Player.log`

What it owns:

- file offsets
- rotation handling
- reading batches of appended text

Why it matters:

- if this is wrong, everything downstream is wrong

`src/manasight_parser_py/log/entry.py`

Purpose:

- group raw lines into meaningful entries

Plain-English job:

- Arena writes many messy lines
- this file decides which lines belong together as one event-sized chunk

Why it matters:

- if entries are grouped badly, even perfect parser modules cannot save you

`src/manasight_parser_py/stream.py`

Purpose:

- connect the tailer to the event bus

Plain-English job:

- keep listening for new log updates
- pass grouped entries into the parser flow

Event Definitions and Routing

`src/manasight_parser_py/events.py`

Purpose:

- define the event types that the parser can emit

Examples:

- GameStateEvent
- ClientActionEvent
- MatchStateEvent
- GameResultEvent
- connection-health events

Why it matters:

- this is the project's vocabulary

`src/manasight_parser_py/router.py`

Purpose:

- ask each parser module whether it recognizes a log entry

Plain-English job:

- "Which parser, if any, knows what this entry means?"

Why it matters:

- this is the dispatch layer between raw entries and typed events

Parser Modules

The parser modules turn recognized MTGA patterns into structured payloads.

Main families:

- `parsers/client_actions.py`
- `parsers/match_state.py`
- `parsers/collection.py`
- `parsers/inventory.py`
- `parsers/rank.py`
- `parsers/connection_*`
- `parsers/gre/`

Why GRE Matters So Much

GRE is the gameplay rules engine side of MTGA's messages.

In this project, the GRE modules are where the really important gameplay information comes from:

- board state
- game identity
- turn information
- object identity
- zone movement
- connect response data

The GRE files:

- `parsers/gre/__init__.py`
- `parsers/gre/game_state.py`
- `parsers/gre/game_result.py`
- `parsers/gre/connect_resp.py`
- `parsers/gre/turn_info.py`

These are some of the most important files if your goal is "understand what cards are being played."

The App Truth Layer

This is the heart of the project.

`src/manasight_parser_py/app/models.py`

Purpose:

- define the normalized shapes for games and matches

Main classes:

- GameSummary
- MatchSummary

Plain-English job:

- give the rest of the code one stable place to store "what we know about this game/match"

Why it matters:

- if the model shape is confused, the rest of the pipeline becomes confused too

`src/manasight_parser_py/app/extractors.py`

Purpose:

- read raw parser payloads and extract reusable facts

Examples:

- match ID
- game number
- turn number
- active player
- local seat/team
- private hand instance IDs
- instance -> grpId lookups

Why it matters:

- this is where raw gameplay payloads become reusable facts

Important concept:

- this file does not decide "what row to send to Sheets"
- it decides "what facts are present in the event"

`src/manasight_parser_py/app/state.py`

Purpose:

- own live match/game interpretation state

Plain-English job:

- remember what is happening across many events over time

Examples of remembered state:

- current match ID

- current game number
- current player team
- opening hand candidates
- mulligan sequence
- whether sideboarding happened
- what match rows were already posted

Why it matters:

- a single MTGA event often does **not** contain the full story
- this file stitches many events together into one coherent match/game story

This is one of the most important truth-producing files in the entire repo.

`src/manasight_parser_py/app/transforms.py`

Purpose:

- decide which parsed events are worth keeping
- convert events into local rows or debug rows
- generate short summaries for logs

Plain-English job:

- act as a filter and serializer

Why it matters:

- not every MTGA event deserves to become a saved row or posted row

Gameplay Action Extraction

`src/manasight_parser_py/app/gameplay_actions.py`

Purpose:

- convert gameplay object changes into card actions

Examples:

- `land_played`
- `spell_cast`
- `spell_finished`
- `permanent_resolved`
- `permanent_left_battlefield`

Why it matters:

- "what card got played?" - "what actually happened during the game?"

- this is the project's main answer to:

How it works in plain English:

- 1 read zones, objects, actions, annotations
- 2 compare current object positions to previous positions
- 3 infer meaningful transitions
- 4 preserve grpId identity
- 5 write action artifacts locally

Important limitation:

- this is still heuristic
- a heuristic is a practical best guess, not a perfect formal guarantee

This is one of the most goal-critical files for your card-analysis ambitions.

Card Identity, Catalogs, and Confirmation Tools

This project has multiple card-identity systems because MTGA and external card data do not line up perfectly.

`src/manasight_parser_py/app/card_catalog.py`

Purpose:

- maintain the Arena-aware card catalog
- sync card metadata from approved data sources

This is your broad card database layer.

`src/manasight_parser_py/app/grp_id_catalog.py`

Purpose:

- maintain a gameplay-first grpId catalog

Why it matters:

- gameplay uses MTGA-native identifiers like grpId
- analytics need stable human-readable card names

This file is the bridge between those worlds.

`src/manasight_parser_py/app/grp_id_candidates.py`

Purpose:

- score likely card identities for unresolved grpIds

`src/manasight_parser_py/app/hand_confirmations.py`

Purpose:

- track manual confirmations from your observed hand/watchlist workflow

`src/manasight_parser_py/app/arena_id_validation.py`

Purpose:

- validate saved logs against the card catalog
- generate evidence for override or contradiction work

Why this whole area exists:

- MTGA identity is messy
- some cards are easy
- some cards need deck context
- some need gameplay evidence
- some need manual confirmation

This is not accidental complexity. It is the actual hard problem.

Match and Card Analytics

`src/manasight_parser_py/app/card_performance.py`

Purpose:

- aggregate local action data and game outcomes into card-level stats

Examples:

- opening-hand win rate
- cast win rate
- seen-in-game win rate
- postboard cast performance
- mulligan tax
- package/co-occurrence summaries

Why it matters:

- this is how the project turns "raw gameplay observations" into "card-level strategic insight"

Important limitation:

- card-performance quality depends on gameplay identity quality
- if grpId resolution is weak, card stats are weaker too

Local Artifacts

`src/manasight_parser_py/app/runtime_surfaces.py`

Purpose:

- build local JSON and Markdown artifacts for current deck, current match, collection, timelines, and history

Examples:

- active match snapshot
- active timeline
- active deck profile
- match history
- collection profile

Why it matters:

- this is the local read-model layer
- a read model is data shaped for reading and reporting, not for raw event ingestion

Important truth:

- these artifacts are incredibly useful
- but they are not all equally good things to live-post to Sheets

Background Optional Work

`src/manasight_parser_py/app/analytics_sidecar.py`

Purpose:

- run optional analytics and export work outside the core parser loop

Examples:

- runtime surface updates
- card-performance refreshes
- optional sheet export families
- startup sync tasks

Why it exists:

- live match parsing should stay fast
- optional analytics should not slow down the parser

Current reality:

- this file is the right architectural idea
- but it still needs careful throttling or it can become too chatty

Webhook Transport

`src/manasight_parser_py/app/outputs.py`

Purpose:

- write local JSONL logs
- queue webhook rows
- send rows to Apps Script

Important design detail:

- this file is transport
- it should not decide gameplay truth

That means:

- if a card name is wrong, fix the parser/state side first
- do not fix it by hacking the webhook layer

Runtime Diagnostics and Local API

`src/manasight_parser_py/app/diagnostics.py`

Purpose:

- runtime log setup
- status file management
- failure records
- webhook success/failure tracking

Why it matters:

- this is observability
- observability means "what evidence do I have about what the system is doing?"

src/manasight_parser_py/app/status_api.py

Purpose:

- expose local status endpoints like /status, /actions, /collection, /match-history

Why it matters:

- local tools can consume these artifacts without reading raw files manually

Why it should probably stay local:

- status data is very useful
- but it is not necessarily spreadsheet-worthy data

Google Sheets Transport

tools/google_apps_script/Code.gs

Purpose:

- receive webhook payloads
- route them to the correct landing tab
- upsert rows in the workbook

What it should be:

- transport and workbook-writing logic

What it should **not** be:

- the place where match truth is invented

This distinction is very important.

If the parser knows the truth, the sheet receiver should mostly just land it cleanly.

What Lives In Google Sheets vs What Should Stay Local

Best always-on sheet tabs

- Match Log
- Game Log
- Dashboard
- Experiments

Good support tabs, but not necessarily live-heavy

- Deck Snapshot
- Card Performance

Better as local files than live sheet feeds

- parser status
- collection snapshot
- raw action stream
- webhook diagnostics
- candidate/debug reports

Why:

- Sheets is best for stable analysis-ready rows
- local files are better for debug-heavy or very chatty data

Current Goals vs Current Alignment

Here is an honest review of where the code is aligned and where it is not.

Aligned With Your Goals

1. Match and game tracking

This is one of the strongest parts of the project.

The combination of:

- `state.py`
- `models.py`
- `extractors.py`
- `runner.py`

is a reasonable foundation for reliable match and game summaries.

2. Local observability

The project is strong at saving local artifacts, logs, and status files.

That makes it much more debuggable than a simple one-file script.

3. grpId-first direction

Moving gameplay interpretation toward MTGA-native grpId identity is the correct long-term direction for your goals.

4. Card-performance direction

The project now has a real path from:

gameplay actions -> per-game observations -> per-card stats

That is exactly what you need for meaningful card evaluation.

Not Fully Aligned Yet

1. Too much optional sheet traffic

This has been one of the clearest misalignments.

The project started generating support/debug sheet traffic that was more expensive than useful during live play.

That is why the recent lean-sheet changes matter.

2. Too much global module state

The project is better than it used to be, but a lot of live state is still held at module level.

That makes it easier to get moving quickly, but harder to reason about long-term.

3. Card identity is still the hardest real problem

The parser can now see much more than before, but:

- unresolved grpIds
- multiface/adventure handling
- opponent-side card naming

are still the places where your long-term goals are most fragile.

4. Some bridge code still exists

You still have "old world / new world / bridge code" overlap.

Examples:

- Apps Script + workbook behaviors that still reflect older project phases
- support artifacts that are useful locally but not ideal as live sheet tabs

Old World, New World, and Bridge Code

This is one of the most important mental models for this repo.

Old world

The older project style was closer to:

- parser output
- immediate sheet posting
- more workbook-side logic

New world

The newer project style is:

- parser/state owns truth
- local JSON/Markdown artifacts are rich and useful
- Sheets receives analysis-ready or summary-ready rows
- `grpId` gameplay identity is first-class

Bridge code

Bridge code is the awkward middle area where both worlds still exist.

Examples:

- Apps Script routing for many row families
- sheet export families that are useful locally but not ideal for live workbook posting
- support tabs that exist partly for debugging and partly for analysis

Bridge code is not "bad." It just means the migration is not fully finished.

What I Think The Project Should Optimize For

If I were turning this into a cleaner long-term tool, I would optimize for:

- 1 **correct local truth**
- 2 **fast live gameplay responsiveness**
- 3 **lean workbook landing tabs**
- 4 **better card identity**
- 5 **stable per-card analytics**

That leads to a simple rule:

```
Keep local truth rich.  
Keep live sheet transport lean.  
Promote only analysis-ready rows to the workbook.
```

Recommended Next Refactors

These are the highest-value refactors from a professional-maintenance standpoint.

1. Keep trimming optional live workbook traffic

Goal:

- only send the sheet rows that directly support your real analysis workflow

2. Keep improving card identity and multiface handling

Goal:

- make per-card analytics more trustworthy

3. Shrink reliance on launcher-side live monitoring

Goal:

- treat the launcher as a controller, not the main runtime dashboard

4. Continue moving from event spam to aggregated facts

Good spreadsheet-facing targets:

- Match Log
- Game Log
- Deck Snapshot
- Card Performance

Less useful live spreadsheet targets:

- parser health
- raw action stream
- frequent snapshot churn

5. Eventually reduce module-global state

Not because it is morally bad, but because it becomes harder to reason about as the project grows.

A Practical Reading Order

If you want to study the code without getting overwhelmed, use this order:

- 1 `main.py`
- 2 `src/manasight_parser_py/app/runner.py`
- 3 `src/manasight_parser_py/app/state.py`
- 4 `src/manasight_parser_py/app/models.py`
- 5 `src/manasight_parser_py/app/extractors.py`
- 6 `src/manasight_parser_py/app/transforms.py`
- 7 `src/manasight_parser_py/app/gameplay_actions.py`
- 8 `src/manasight_parser_py/app/runtime_surfaces.py`
- 9 `src/manasight_parser_py/app/analytics_sidecar.py`
- 10 `src/manasight_parser_py/app/outputs.py`

```
11 tools/google_apps_script/Code.gs
12 tools/auto_launcher/manasight_launcher_auto.py
```

If you want to understand raw parsing more deeply after that:

```
1 src/manasight_parser_py/router.py
2 src/manasight_parser_py/parsers/gre/
3 src/manasight_parser_py/log/entry.py
4 src/manasight_parser_py/log/tailer.py
```

What The Project Can Already Do Well

- parse live MTGA logs
- build normalized match and game summaries
- archive local event truth
- extract gameplay action logs
- track active submitted decks
- maintain a grpId-first identity layer
- generate per-card performance artifacts
- feed Google Sheets with analysis rows

That is already a lot. This is not a toy script anymore.

What Still Needs The Most Attention

- card identity edge cases
- multiface/adventure correctness
- keeping optional analytics from slowing live play
- reducing bridge-code complexity
- keeping workbook usage aligned with actual analysis needs

Final Mental Model

If I had to give you one final mental model for the whole repo, it would be this:

```
Manasight+ is not one script.
It is a pipeline of truth, memory, transport, and reporting.
```

More specifically:

```
raw log
-> typed events
```

```
-> match/game truth  
-> local artifacts  
-> optional transport  
-> workbook reporting
```

When the system behaves strangely, ask:

- 1 did the raw log contain the fact?
- 2 did the parser recognize it?
- 3 did state remember it correctly?
- 4 did the local artifact preserve it?
- 5 did the transport send it?
- 6 did the workbook land it?

That question ladder will save you a huge amount of debugging time.

If You Want One More Document After This

The next most useful document would be a **file-by-file debugging playbook** that says:

- if Match Log is wrong, read these 5 files first
- if card identity is wrong, read these 6 files first
- if the workbook is stale, inspect these transport layers first
- if performance is bad, inspect these hot-path files first

That would be a good follow-up once you finish reading this guide.